

OMS Core

A full-stack order management system for small e-commerce businesses, with AI-assisted product listing and invoice parsing

Project Status: Functional Prototype — Emergent Platform Date: July–August 2026

Summary

A working web-based order management system built on the Emergent AI development platform. The application covers inventory management, supplier tracking, stock receiving, barcode scanning, AI-generated product copy, AI invoice parsing, and a KPI dashboard. Backend tests pass 16/16. Channel synchronisation with external marketplaces is simulated; no real marketplace API connections exist.

The Problem

Small and medium e-commerce businesses need affordable tools to manage inventory, suppliers, and orders — but enterprise OMS solutions are prohibitively expensive, over-engineered for small catalogues, and rarely include AI-assisted content tools. Businesses operating across multiple sales channels (Shopify, Amazon, Etsy, eBay) also lack a unified view of their inventory position.

Objective

Build a full-stack OMS MVP that covers the core operational cycle: stock receipt, inventory tracking, order creation, and sales reporting — augmented with two AI features (listing copy generation and invoice parsing) and a multi-channel sync status view. Delivered as a working prototype on the Emergent cloud platform.

Intended Users

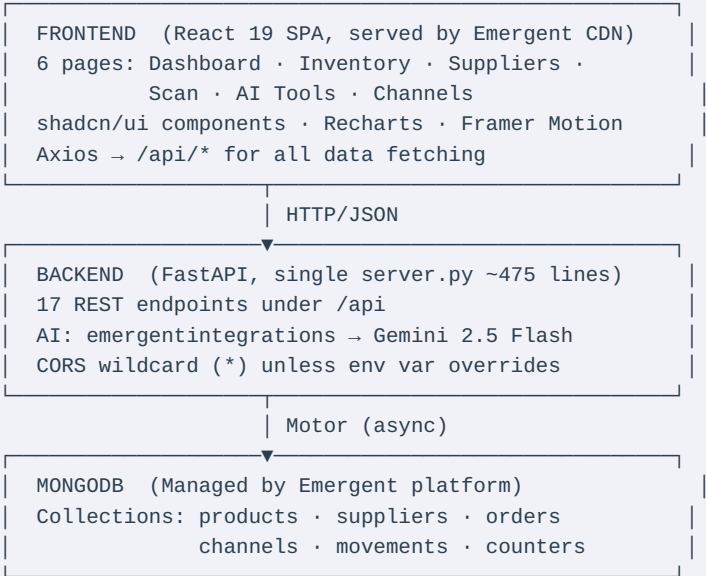
A small-to-medium e-commerce business owner or operator managing a physical product catalogue across multiple online channels. The application is single-user by design (no authentication layer); it is not architected for team or multi-tenant use.

Why This Matters

This project illustrates a practical pattern in modern software development: using an AI code-generation platform (Emergent) to produce a functional, testable prototype rapidly, with the human contributor acting as product designer, requirements author, and acceptance tester. The integration of Gemini 2.5 Flash for multimodal tasks (reading invoice images, generating listing copy) demonstrates how AI capabilities can be embedded into operational business tools without specialist ML expertise. The project also surfaces real constraints of this approach — platform lock-in, proprietary dependencies, and the gap between simulated and real integrations.

Architecture and End-to-End Workflow

Three-tier web application on the Emergent cloud platform:



EMERGENT PLATFORM manages: hosting, MongoDB, secrets, base container (fastapi_react_mongo_shadcn_base_image).

Key API endpoints:

GET/POST/PUT/DELETE	/api/products	Inventory CRUD
GET/POST/PUT/DELETE	/api/suppliers	Supplier CRUD
POST	/api/stock-in	Receive shipment
GET/POST	/api/orders	Record sale / list orders
GET	/api/channels	Channel sync status
POST	/api/channels/sync	Trigger (simulated) sync
GET	/api/alerts/low-stock	Reorder alerts
GET	/api/dashboard	KPI aggregates + chart data
POST	/api/ai/copywriter	Gemini listing copy
POST	/api/ai/parse-invoice	Gemini invoice parsing

Technologies Used

Technology	Version	Role
Python / FastAPI	0.110.1	Backend API and business logic
React	19.0.0	Frontend SPA
MongoDB / Motor	motor 3.3.1	Primary data store (managed by Emergent)
Gemini 2.5 Flash	via emergentintegrations 0.2.0	AI listing copy + invoice parsing
shadcn/ui	Radix UI primitives	Full component library
Recharts	3.6.0	Bar chart and pie chart on dashboard
Framer Motion	11.18.0	Page transitions, scan result animation
Tailwind CSS	3.4.17	Utility-first styling
Axios	1.18.0	HTTP client
pytest	>=8.0.0	Backend integration tests (16/16 pass)
Emergent platform	Base image: release-25072026-1	Cloud hosting, managed DB, secrets

What Was Actually Built

Each capability is independently labelled based on evidence found in the repository.

Capability	Status
KPI Dashboard (Active Orders, Revenue, Low Stock, Units)	Implemented
Stock-on-hand bar chart (Recharts)	Implemented
Collection-mix pie chart (Recharts)	Implemented
Reorder alerts list on dashboard	Implemented
Product CRUD with auto-sequential SKU (OMS-001...)	Implemented
Product search by name or SKU	Implemented
Master-detail inventory view	Implemented
Record Sale (one-click: creates order, decrements stock)	Implemented
Stock status badges (Out of stock / Low / Ready to sell)	Implemented
Supplier CRUD with auto-ID (OMS-SUP-XXXXXX)	Implemented
Stock-In / Receive Shipment — manual SKU entry	Implemented
Stock-In — camera barcode scan (BarcodeDetector API)	Implemented with limitations
Stock movement log (backend: movements collection)	Implemented
AI Listing Writer — title, description, SEO keywords, care tip (Gemini)	Implemented
AI Invoice Reader — PDF/image → structured JSON (Gemini multimodal)	Implemented
Five-channel sync status display	Implemented
Multi-channel marketplace sync (Shopify / Amazon / Etsy / eBay / Social)	Implemented with limitations
Responsive layout (desktop sidebar / mobile tab bar)	Implemented
Movement history view (frontend)	Not present
Real marketplace OAuth/API integration	Planned
Email reorder alerts	Planned
User authentication / multi-user support	Planned
Barcode label printing	Planned
Sales analytics over time	Planned
3PL shipping routing	Planned

Implementation Notes

The codebase was generated by the Emergent AI platform; all six git commits carry the author 'emergent-agent-e1 <github@emergent.sh>'. Development spanned four days (25–29 July 2026). The backend is a single-file FastAPI application. The frontend follows standard Create React App conventions with shadcn/ui components throughout.

The AI features rely on 'emergentintegrations==0.2.0', a proprietary Python library specific to the Emergent platform. This library wraps the Gemini API; replacing it with the standard Google AI SDK would require a modest rewrite of the two AI endpoints. The channel sync feature updates MongoDB timestamps and item counts only — no network calls are made to any marketplace. The UI footer explicitly labels this 'running in demo mode'. Several dependencies installed by the platform scaffold (boto3, bcrypt, passlib, pyjwt) are not used anywhere in the application code. Camera barcode scanning uses the browser-native BarcodeDetector API, which is supported in Chromium-based browsers only; the application degrades gracefully to manual entry.

User Contribution and Role

The user's contribution to OMS Core is that of product designer, requirements author, and acceptance tester. The PRD.md (memory/PRD.md) captures the product requirements. The user specified the feature set, validated the acceptance criteria, and directed the Emergent platform through iterative build cycles. All application code was generated by the Emergent AI agent. This reflects an increasingly common pattern in commercial product development where human contribution centres on product direction and quality assurance rather than line-by-line implementation.

Testing and Validation Evidence

Backend integration tests: 16/16 pass (verified by JUnit XML output at test_reports/pytest/pytest_results.xml, timestamp 2026-07-29T16:27:30, 10.765 s total duration). Tests cover all major API flows: product CRUD, supplier CRUD, stock receipt, order creation (including oversell rejection), dashboard shape, low-stock alerts, both AI endpoints (live Gemini calls, 4.06 s and 2.71 s respectively), and channel seeding.

A UI test summary (test_reports/iteration_1.json) reports 100% pass on Playwright flows covering all six pages. However, the Playwright test scripts themselves are not present in the repository, so this result cannot be independently re-run from the committed code alone.

Known Limitations

- No authentication — all API routes are publicly accessible; CORS defaults to wildcard (*). Unsuitable for production without adding an auth layer.
- Channel sync is entirely simulated — the five marketplace channels only update MongoDB timestamps; no real API calls are made to Shopify, Amazon, Etsy, eBay, or Social Shop.
- Proprietary dependency — emergentintegrations==0.2.0 is specific to the Emergent platform. Running outside that environment requires rewriting the two AI endpoints.
- No self-hosting path — there is no Dockerfile, docker-compose, or CI/CD configuration. Deployment requires the Emergent platform.
- No image upload — product images are URL strings only; no file upload or CDN integration.
- No pagination — product queries cap at 2,000 documents; order queries at 500. Suitable for small catalogues; not architected for scale.
- Camera barcode scanning requires a Chromium-based browser (BarcodeDetector API); degrades to manual entry on other browsers.

- No order status management — orders are created as 'Active' and never transition (no fulfilment, shipping, or cancellation flow).
- Playwright test scripts not committed — the UI test pass-rate cannot be independently verified from the repository.

Lessons Learned

AI code-generation platforms can produce functional, well-structured applications quickly when requirements are clearly specified. The test evidence (16/16 backend tests passing) demonstrates that generated code can meet concrete acceptance criteria. However, the experience also illustrates the importance of scrutinising proprietary dependencies: the emergentintegrations library creates platform lock-in that would require deliberate effort to remove. Simulated integrations (the channel sync feature) can pass superficial tests while masking the significant engineering work still required for a production-ready implementation.

Practical Next Steps

1. Extract from the Emergent platform to a standard Docker deployment (FastAPI + React + MongoDB on docker-compose).
2. Replace emergentintegrations with the standard Google AI Python SDK for Gemini access.
3. Implement an authentication layer (JWT or session-based) before any production exposure.
4. Wire one real marketplace API (Shopify is the best-documented starting point) to validate the channel sync architecture.
5. Add proper pagination to product and order endpoints (limit/offset or cursor-based).
6. Commit the Playwright test scripts to the repository to make UI tests independently reproducible.
7. Add a product image upload endpoint backed by cloud storage (S3 or GCS).

Evidence Base Reviewed

- backend/server.py (~475 lines, FastAPI — read in full)
- backend/tests/backend_test.py (16 pytest integration tests — read in full)
- frontend/src/pages/*.jsx (all six page components — read in full)
- frontend/src/lib/apiClient.js (Axios wrapper — read in full)
- test_reports/pytest/pytest_results.xml (JUnit XML, 16/16 pass, 2026-07-29)
- test_reports/iteration_1.json (UI test summary — Playwright scripts not in repo)
- memory/PRD.md (product requirements and build log)
- design_guidelines.json (design system specification)
- frontend/package.json (exact dependency versions)
- backend/requirements.txt (Python dependencies)
- .emergent/emergent.yml (platform config, base image, job ID)
- git log — 6 commits, 2026-07-25 to 2026-08-13, all by emergent-agent-e1